Assignment 2 Report - Fitting and Alignment

Name: W.A.N.C. Wickramarathna

Index Number: 258844D

Github URL: https://github.com/nishanc/Computer-Vision-A2

1. Line Fitting with Total Least Squares and RANSAC

The dataset in lines.csv contains three point groups, each sampled from a line. The first task was to fit a line to only the first group using total least squares (TLS). TLS was preferred because it accounts for error in both x and y coordinates.

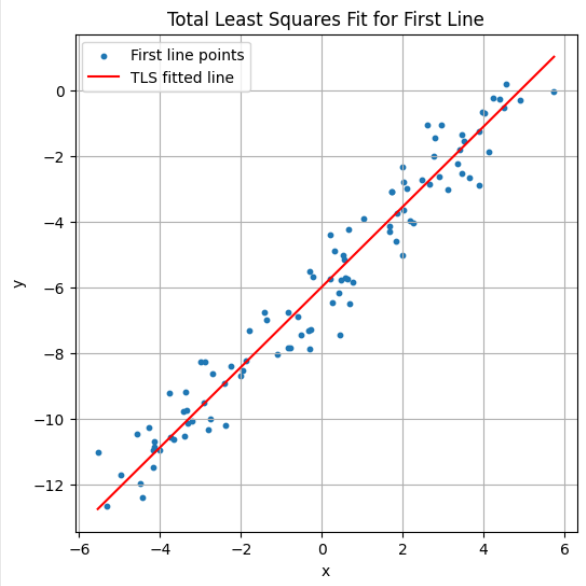
```
def fit_tls_line(x, y):
    points = np.column_stack((x, y))
    centroid = points.mean(axis=0)
    centered = points - centroid
    _, _, vt = np.linalg.svd(centered)
    a, b = vt[-1]
    c = -(a*centroid[0] + b*centroid[1])
    return a, b, c
```

TLS result for the first line: 0.773562x - 0.633721y - 3.794192 = 0, *or equivalently* y = 1.220666x - 5.987165.

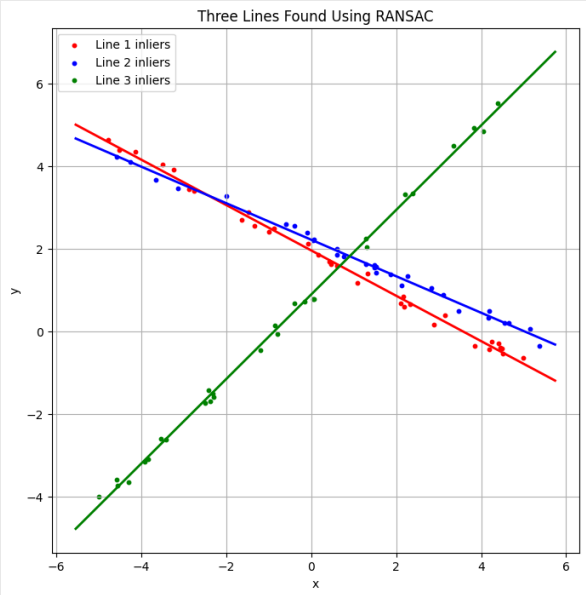
Using all points together, RANSAC was applied three times. After each line was found, its inlier points were removed and the process was repeated on the remaining points. This separated the three line structures successfully.

Detected line	Model	Inliers
Line 1	y = -0.549082x + 1.967696	33
Line 2	y = -0.442100x + 2.226328	30
Line 3	y = 1.023228x + 0.898633	28

Interpretation. The repeated RANSAC strategy produced two negative-slope lines and one positive-slope line, which agrees with the scatter plot. The inlier sets cluster tightly around the fitted lines, showing that the method separated overlapping structures reliably.



Q1-A: Scatter of first-line points with TLS fitted line.



Q1-B: Final RANSAC plot showing the three detected lines and inlier groups.

2. Estimating the Real Size of Earrings from an Image

The earrings were segmented from the light background in HSV color space. A threshold on the saturation channel isolated the gold regions because the background had low saturation. Morphological opening and closing were then used to clean the mask before measuring bounding boxes.

```
img_hsv = cv.cvtColor(img, cv.COLOR_BGR2HSV)
H, S, V = cv.split(img_hsv)
mask = (S > 35).astype(np.uint8) * 255
kernel = np.ones((5,5), np.uint8)
mask = cv.morphologyEx(mask, cv.MORPH_OPEN, kernel)
mask = cv.morphologyEx(mask, cv.MORPH_CLOSE, kernel)
```

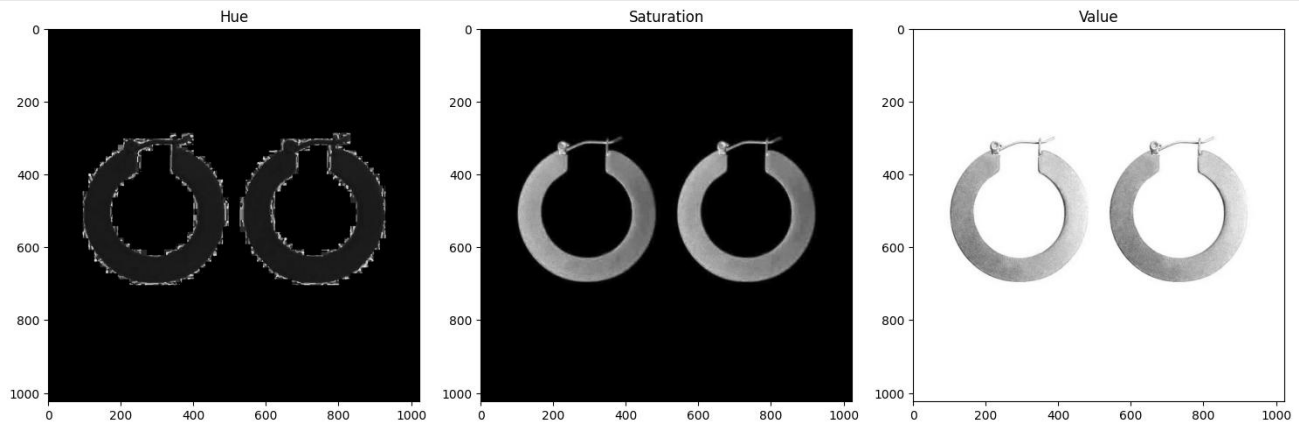
Detected pixel sizes: Earring 1 = 382 × 398 px, Earring 2 = 382 × 399 px.

A pinhole camera model was used to convert pixels to millimetres. With a focal length of 8 mm, pixel size 2.2 µm and object distance 720 mm, the scale became 0.198 mm per pixel.

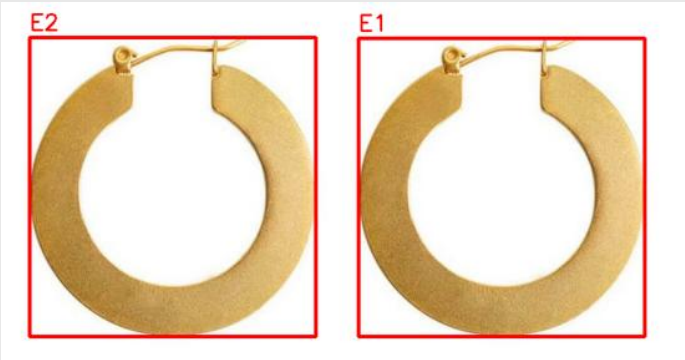
```
pixel_size_mm = 2.2e-3
f_mm = 8.0
Z_mm = 720.0
scale_mm_per_pixel = (pixel_size_mm * Z_mm) / f_mm
```

Earring	Width (mm)	Height (mm)
1	75.64	78.80
2	75.64	79.00

Interpretation. Both earrings have almost identical size, which is expected for a pair. The measured heights are slightly larger than the widths because the bounding box includes the upper hook area as well as the circular body.



Q2-A: HSV channel view.



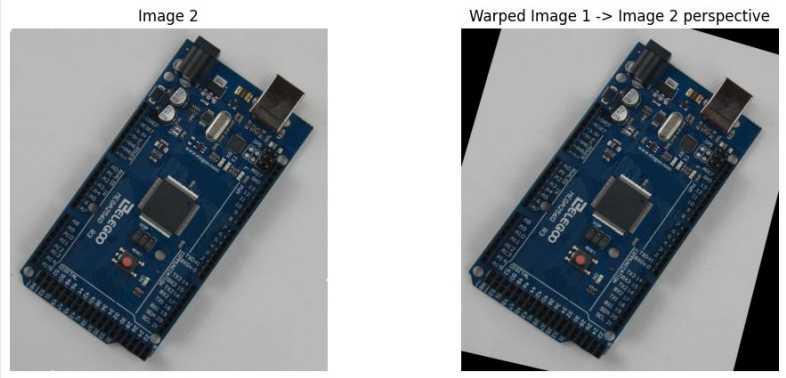
Q2-B: Binary mask and bounding-box result used for measurement.

3. Circuit Board Alignment Using Homography

Two views of a circuit board were aligned using projective transformation. In the manual approach, six corresponding points were clicked on both images and used to estimate a homography. Warping one board into the perspective of the other made it possible to compare them directly by image subtraction.

```
H_manual, _ = cv.findHomography(p1, p2, method=0)
warped_manual = cv.warpPerspective(img1, H_manual, (img2.shape[1], img2.shape[0]))
diff_manual = cv.absdiff(img2, warped_manual)
```

Manual homography discussion. The warped board was aligned reasonably well, but the manual-point difference image still contained bright edges and visible board structure. This indicates that small point-selection errors propagated into the homography and prevented perfect alignment.



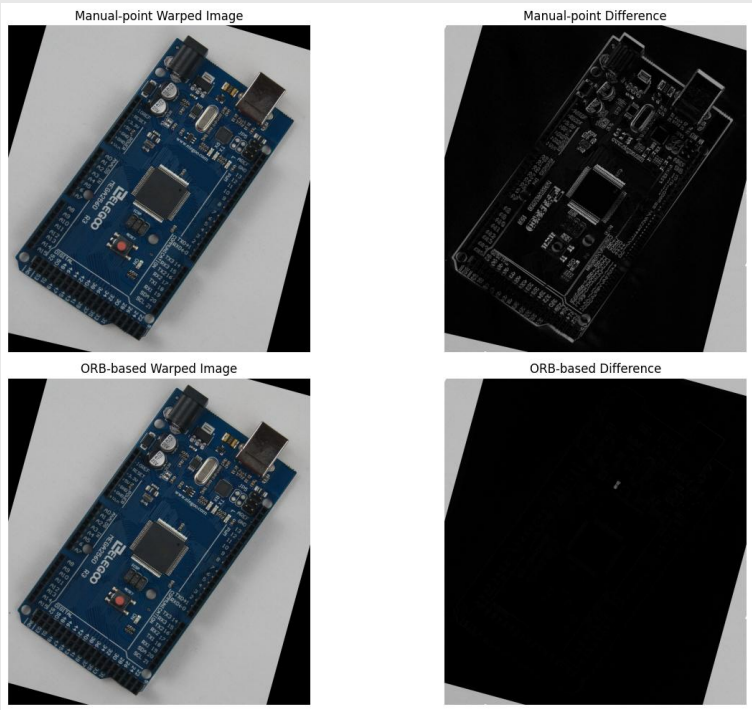
Q3-A: Manual-point warped image and manual-point difference image.

An automatic alternative was then implemented using ORB keypoints and descriptor matching. ORB produced many correspondences, and RANSAC inside homography estimation reduced the effect of mismatched points.

```
orb = cv.ORB_create(1500)
kp1, des1 = orb.detectAndCompute(gray1, None)
kp2, des2 = orb.detectAndCompute(gray2, None)
matches = bf.match(des1, des2)
H_orb, _ = cv.findHomography(src_pts, dst_pts, cv.RANSAC, 5.0)
```

ORB matching statistics. 1500 keypoints were detected in each image, and 1141 descriptor matches were obtained.

Interpretation. The ORB-based warped image aligned more accurately with the target image. Its difference image was much darker than the manual version, showing that the transformation was closer to the correct projective mapping.



Q3-B: Manual difference image and ORB-based difference image.